JAVASCRIPT:

JavaScript, often abbreviated as JS, is a high-level, interpreted programming language primarily known for its role in making web pages interactive and dynamic.

The programs in this language are called *scripts*. They can be written right in a web page's HTML and run automatically as the page loads.

A JavaScript Virtual Machine (JS VM) is a program that executes JavaScript code. It acts as an interpreter and runtime environment, translating high-level JavaScript code into low-level machine code that the computer's hardware can understand and execute.

Examples of JavaScript VMs/Engines:

- V8: Used in Google Chrome, Microsoft Edge, and Node.js.
- SpiderMonkey: Used in Mozilla Firefox.
- JavaScriptCore (Nitro): Used in Apple Safari.

JS naming principle

JavaScript file naming conventions are important for maintaining consistency, readability, and project organization. While there isn't one single, universally mandated convention, several common practices are widely adopted:

General Principles:

- Consistency: The most crucial aspect is to maintain a consistent naming style throughout your entire project.
- Readability: File names should clearly and concisely describe the file's content or purpose.
- Lowercase: File names should generally be in lowercase.
- No Spaces: Avoid spaces in file names; use hyphens or underscores instead.

- .js Extension: All JavaScript files must have the .js extension.

Note : In HTML, JavaScript code is inserted between <script> and </script> tags.

Basic program

Node JS installation steps

Download: Visit the official Node.js website (nodejs.org) and download the appropriate installer for your operating system. It is generally recommended to choose the LTS (Long Term Support) version for stability.

Run the Installer: Execute the downloaded installer file (e.g., .msi for Windows, .pkg for macOS).

Follow Setup Wizard: Follow the on-screen instructions, accepting the license agreement and choosing the installation location (keeping the default is usually fine). Ensure that the option to add Node.js to your system's PATH is selected, as this allows you to run Node.js commands from any terminal or command prompt.

Complete Installation: Click "Install" and then "Finish" once the process is complete.

Verify Installation: Open a new terminal or command prompt and type the following commands to verify that Node.js and npm (Node Package Manager, which is bundled with Node.js) are installed correctly:

Code

```
node -v
npm -v
```

Note : Please run javascript program in integrated terminal in VS Code

JavaScript Statements

A computer program is a list of "instructions" to be "executed" by a computer.

These programming instructions are called statements.

Most JavaScript programs contain many statements.

The statements are executed, one by one, in the same order as they are written.

Semicolons separate JavaScript statements.

Add a semicolon at the end of each executable statement:

JavaScript Comments

JavaScript comments can be used to explain JavaScript code, and to make it more readable.

JavaScript comments can also be used to prevent execution, when testing alternative code.

Single Line Comments

Single line comments start with //.

Any text between // and the end of the line will be ignored by JavaScript (will not be executed).

This example uses a single-line comment before each code line:

```
let x = 5;      // Declare x, give it the value of 5

let y = x + 2;  // Declare y, give it the value of x + 2
```

Multi-line Comments

Multi-line comments start with /* and end with */.

Any text between /* and */ will be ignored by JavaScript.

This example uses a multi-line comment (a comment block) to explain the code:

JavaScript Variables

JavaScript variables are containers for data.

Variables are identified with unique names called identifiers.

Names can be short like x, y, z.

Names can be descriptive age, sum, carName.

The rules for constructing names (identifiers) are:

- Names can contain letters, digits, underscores, and dollar signs.
- Names must begin with a letter, a $ sign or an underscore (_).
- Names are case sensitive (X is different from x).
- Reserved words (JavaScript keywords) cannot be used as names.

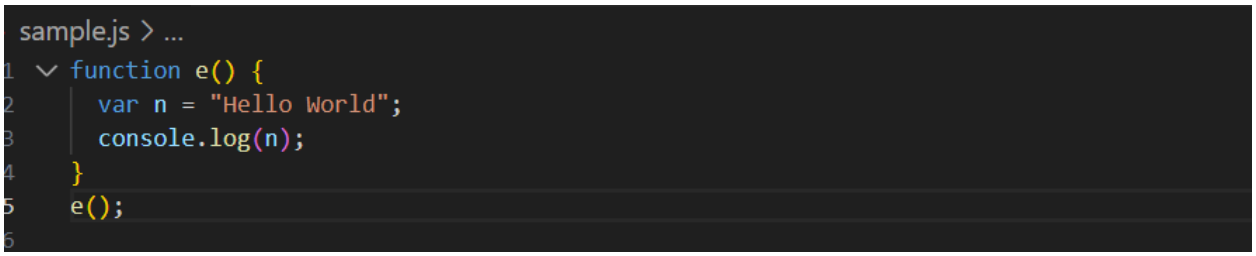
JavaScript provides three ways to declare variables: var, let, and const, but they differ in scope, hoisting behaviour, and re-assignment rules.

- var: Declares variables with function or global scope and allows re-declaration and updates within the same scope.
- let: Declares variables with block scope, allowing updates but not re-declaration within the same block.

- const: Declares block-scoped variables that cannot be reassigned after their initial assignment.

1. Declaring Variables with var

var is the original keyword for declaring variables in JavaScript. It is function-scoped or globally scoped, depending on where it's declared.



Open integrated terminal for sample.js file and run node sample.js

Output

Hello world

- The function e declares a variable n with the value "Hello world" and logs it to the console.
- When the function e() is called, it outputs "Hello world" to the console.

2. Block Scope with let

Introduced in ES6, let provides block-level scoping. This means the variable is only accessible within the block (like loops or conditionals) where it is declared.

```
if (true) {
    let age = 30;
```

```
    console.log(age);
}
console.log(age)
```

Output

```
ReferenceError: age is not defined
    at Object.<anonymous> (/home/guest/sandbox/Solution.js:5:13)
    at Module._compile (node:internal/modules/cjs/loader:1198:14)
    at Object.Module._extensions..js (node:internal/modules/cjs/loader:
    at Module.load (node:internal/modules/cjs/loader:1076:32)
    at Function.Module._load (node:internal/modules/cjs/loader:911:12)
    at Function.executeUserEntryPoint [...
```

- Block Scope: The variable age is declared with let inside the if block, so it is only accessible within that block and cannot be accessed outside of it.
- Reference Error: The second console.log(age) will throw a Reference Error because age is not defined in the outer scope.

3. const

const is used to declare variables that should not be reassigned after their initial assignment. This keyword is also block-scoped like let.

```
const country = "USA";
console.log(country);
```

Output

```
USA
```

- The variable country is declared with const, meaning its value cannot be reassigned after initialization.
- The value of country (which is "USA") is printed to the console.

// JS Data Types

Data types

A value in JavaScript is always of a certain type. For example, a string or a number.

There are eight basic data types in JavaScript. Here, we'll cover them in general and in the next chapters we'll talk about each of them in detail.

We can put any type in a variable. For example, a variable can at one moment be a string and then store a number:

```
// no error

let message = "hello";


message = 123456;
```

Programming languages that allow such things, such as JavaScript, are called "dynamically typed", meaning that there exist data types, but variables are not bound to any of them.

Primitive Data types

Number

```
let n = 123;
```

```
n = 12.345;
```

The *number* type represents both integer and floating point numbers.

There are many operations for numbers, e.g. multiplication `*`, division `/`, addition `+`, subtraction `-`, and so on.

Besides regular numbers, there are so-called "special numeric values" which also belong to this data type: `Infinity`, `-Infinity` and `NaN`.

> `Infinity` represents the mathematical Infinity ∞. It is a special value that's greater than any number.
>
> We can get it as a result of division by zero:
>
> ```
> alert( 1 / 0 ); // Infinity
> ```
>
> Or just reference it directly:
>
> ```
> alert( Infinity ); // Infinity
> ```
>
> `NaN` represents a computational error. It is a result of an incorrect or an undefined mathematical operation, for instance:
>
> ```
> alert( "not a number" / 2 ); // NaN, such division is erroneous
> ```
>
> `NaN` is sticky. Any further mathematical operation on `NaN` returns `NaN`:

```
alert( NaN + 1 ); // NaN
```

```
alert( 3 * NaN ); // NaN
```

```
alert( "not a number" / 2 - 1 ); // NaN
```

So, if there's a `NaN` somewhere in a mathematical expression, it propagates to the whole result (there's only one exception to that: `NaN ** 0` is `1`).

Mathematical operations are safe

Doing maths is "safe" in JavaScript. We can do anything: divide by zero, treat non-numeric strings as numbers, etc.

The script will never stop with a fatal error ("die"). At worst, we'll get `NaN` as the result.

Special numeric values formally belong to the "number" type. Of course they are not numbers in the common sense of this word.

We'll see more about working with numbers in the chapter Numbers.

BigInt

In JavaScript, the "number" type cannot safely represent integer values larger than $(2^{53}-1)$ (that's `9007199254740991`), or less than $-(2^{53}-1)$ for negatives.

To be really precise, the "number" type can store larger integers (up to $1.7976931348623157 * 10^{308}$), but outside of the safe integer range $\pm(2^{53}-1)$

there'll be a precision error, because not all digits fit into the fixed 64-bit storage. So an "approximate" value may be stored.

For example, these two numbers (right above the safe range) are the same:

```
console.log(9007199254740991 + 1); // 9007199254740992
```

```
console.log(9007199254740991 + 2); // 9007199254740992
```

So to say, all odd integers greater than $(2^{53}-1)$ can't be stored at all in the "number" type.

For most purposes $\pm(2^{53}-1)$ range is quite enough, but sometimes we need the entire range of really big integers, e.g. for cryptography or microsecond-precision timestamps.

`BigInt` type was recently added to the language to represent integers of arbitrary length.

A `BigInt` value is created by appending `n` to the end of an integer:

```
// the "n" at the end means it's a BigInt
```

```
const bigInt = 1234567890123456789012345678901234567890n;
```

As `BigInt` numbers are rarely needed, we don't cover them here, but devoted them a separate chapter BigInt. Read it when you need such big numbers.

String

A string in JavaScript must be surrounded by quotes.

```
let str = "Hello";
```

```
let str2 = 'Single quotes are ok too';


let phrase = `can embed another ${str}`;
```

In JavaScript, there are 3 types of quotes.

1. Double quotes: `"Hello"`.
2. Single quotes: `'Hello'`.
3. Backticks: `` `Hello` ``.

Double and single quotes are “simple” quotes. There’s practically no difference between them in JavaScript.

Backticks are “extended functionality” quotes. They allow us to embed variables and expressions into a string by wrapping them in `${…}`, for example:

```
let name = "John";


// embed a variable

alert( `Hello, ${name}!` ); // Hello, John!


// embed an expression


alert( `the result is ${1 + 2}` ); // the result is 3
```

The expression inside `${…}` is evaluated and the result becomes a part of the string. We can put anything in there: a variable like `name` or an arithmetical expression like `1 + 2` or something more complex.

Please note that this can only be done in backticks. Other quotes don't have this embedding functionality!

```
alert( "the result is ${1 + 2}" ); // the result is ${1 + 2} (double quotes do nothing)
```

Boolean (logical type)

The boolean type has only two values: `true` and `false`.

This type is commonly used to store yes/no values: `true` means "yes, correct", and `false` means "no, incorrect".

For instance:

```
let nameFieldChecked = true; // yes, name field is checked
```

```
let ageFieldChecked = false; // no, age field is not checked
```

Boolean values also come as a result of comparisons:

```
let isGreater = 4 > 1;
```

```
alert( isGreater ); // true (the comparison result is "yes")
```

he "null" value

The special `null` value does not belong to any of the types described above.

It forms a separate type of its own which contains only the `null` value:

```
let age = null;
```

In JavaScript, `null` is not a “reference to a non-existing object” or a “null pointer” like in some other languages.

It’s just a special value which represents “nothing”, “empty” or “value unknown”.

The code above states that `age` is unknown.

The “undefined” value

The special value `undefined` also stands apart. It makes a type of its own, just like `null`.

The meaning of `undefined` is “value is not assigned”.

If a variable is declared, but not assigned, then its value is `undefined`:

```
let age;
```

```
alert(age); // shows "undefined"
```

Technically, it is possible to explicitly assign `undefined` to a variable:

```
let age = 100;
```

```
// change the value to undefined
age = undefined;
```

```
alert(age); // "undefined"
```

…But we don't recommend doing that. Normally, one uses `null` to assign an "empty" or "unknown" value to a variable, while `undefined` is reserved as a default initial value for unassigned things.

Non Primitive Data types

The `object` type is special.

All other types are called "primitive" because their values can contain only a single thing (be it a string or a number or whatever). In contrast, objects are used to store collections of data and more complex entities.

The `symbol` type is used to create unique identifiers for objects. We have to mention it here for the sake of completeness, but also postpone the details till we know objects.

// JS Data types end

```
// JS condition
```

Conditional Statements

When you write code, you often want to perform different actions for different conditions.

Conditional statements run different code depending on a true or false condition.

Conditional statements include:

- if
- if...else
- if...else if...else
- switch
- ternary (? :)

When to use Conditionals

- Use `if` to specify a code block to be executed, if a specified condition is `true`
- Use `else` to specify a code block to be executed, if the same condition is `false`
- Use `else if` to specify a new condition to test, if the first condition is `false`
- Use `switch` to specify many alternative code blocks to be executed
- Use `(? :)` (ternary) as a shorthand for `if...else`
-

The if Statement

Use `if` to specify a **code block** to be executed, if a specified condition is `true`.

Syntax

```
if (condition) {
  // code to execute if the condition is true
}
```

The else Statement

Use `else` to specify a **code block** to be executed, if the same condition is `false`.

Syntax

```
if (condition) {
  // code to execute if the condition is true
} else {
  // code to execute if the condition is false
}
```

The else if Statement

Use `else if` to specify a **new condition** to test, if the first condition is `false`.

Syntax

```
if (condition1) {
  // code to execute if condition1 is true
} else if (condition2) {
  // code to execute if the condition1 is false and condition2 is true
} else {
  // code to execute if the condition1 is false and condition2 is false
}
```

The switch Statement

Use `switch` to specify many **alternative code blocks** to be executed.

Syntax

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

Ternary Operator (? :)

Use `(? :)` (ternary) as a **shorthand** for `if...else`.

Example

```
condition ? expression1 : expression2
```

```
// JS condition ends
```

```
// JS Loops
```

JAVAScript Loops:

The “while” loop

The `while` loop has the following syntax:

```
while (condition) {
  // code
  // so-called "loop body"


}
```

While the `condition` is truthy, the `code` from the loop body is executed.

For instance, the loop below outputs `i` while `i < 3`:

```
let i = 0;
while (i < 3) { // shows 0, then 1, then 2
  alert( i );
  i++;


}
```

A single execution of the loop body is called *an iteration*. The loop in the example above makes three iterations.

If `i++` was missing from the example above, the loop would repeat (in theory) forever. In practice, the browser provides ways to stop such loops, and in server-side JavaScript, we can kill the process.

Any expression or variable can be a loop condition, not just comparisons: the condition is evaluated and converted to a boolean by `while`.

The “do…while” loop

The condition check can be moved *below* the loop body using the `do..while` syntax:

```
do {
  // loop body


} while (condition);
```

The loop will first execute the body, then check the condition, and, while it’s truthy, execute it again and again.

For example:

```
let i = 0;
do {
  alert( i );
  i++;
```

```
} while (i < 3);
```

This form of syntax should only be used when you want the body of the loop to execute **at least once** regardless of the condition being truthy. Usually, the other form is preferred: `while(…) {…}`.

The “for” loop

The `for` loop is more complex, but it’s also the most commonly used loop.

It looks like this:

```
for (begin; condition; step) {
  // ... loop body ...


}
```

Let’s learn the meaning of these parts by example. The loop below runs `alert(i)` for `i` from `0` up to (but not including) `3`:

```
for (let i = 0; i < 3; i++) { // shows 0, then 1, then 2
  alert(i);


}
```

Let’s examine the `for` statement part-by-part:

part		
begin	`let i = 0`	Executes once upon entering the loop.

condition	`i < 3`	Checked before every loop iteration. If false, the loop stops.
body	`alert(i)`	Runs again and again while the condition is truthy.
step	`i++`	Executes after the body on each iteration.

The general loop algorithm works like this:

```
Run begin
→ (if condition → run body and run step)
→ (if condition → run body and run step)
→ (if condition → run body and run step)


→ ...
```

That is, `begin` executes once, and then it iterates: after each `condition` test, `body` and `step` are executed.

If you are new to loops, it could help to go back to the example and reproduce how it runs step-by-step on a piece of paper.

Here's exactly what happens in our case:

```
// for (let i = 0; i < 3; i++) alert(i)

// run begin
let i = 0
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
```

```
  // ...finish, because now i == 3
```

```
// JS Loops END
```

JavaScript Strings

Strings are for storing text

Strings are written with quotes

Strings created with single or double quotes work the same.

There is no difference between the two.

```
let answer1 = "It's alright";
let answer2 = "He is called 'Johnny'";
let answer3 = 'He is called "Johnny"';
```

String Length

To find the length of a string, use the built-in `length` property:

```
let text = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```

```
let length = text.length;
```

String Methods

JavaScript String charAt()

The `charAt()` method returns the character at a specified index (position) in a string:

JavaScript String concat()

`concat()` joins two or more strings:

```
JS sample.js
JS sample.js > ...
1   let text1 = "Hello";
2   let text2 = "World";
3   let text3 = text1.concat(" ", text2);
4   console.log(text3);
5
```

Converting to Upper and Lower Case

A string is converted to upper case with `toUpperCase()`:

A string is converted to lower case with `toLowerCase()`:

```
JS sample.js > ...
1   let text1 = "Hello World!";
2   console.log(text1.toLocaleLowerCase());
3   console.log(text1.toLocaleUpperCase());
4
```

JavaScript String repeat()

The `repeat()` method returns a string with a number of copies of a string.

The `repeat()` method returns a new string.

The `repeat()` method does not change the original string.

JavaScript String includes()

The `includes()` method returns `true` if a string contains a specified string.

Otherwise it returns `false`.

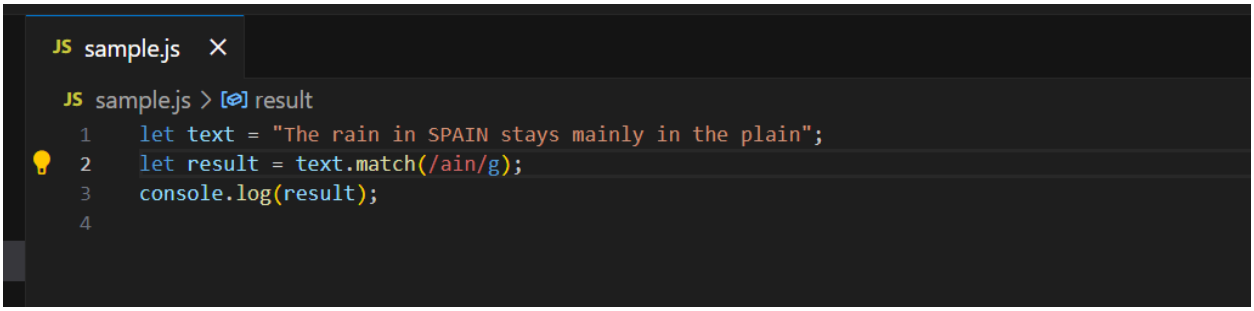
The `includes()` method is case sensitive.

JavaScript String match()

The `match()` method matches a string against a regular expression **

The `match()` method returns an array with the matches.

The `match()` method returns *null* if no match is found.

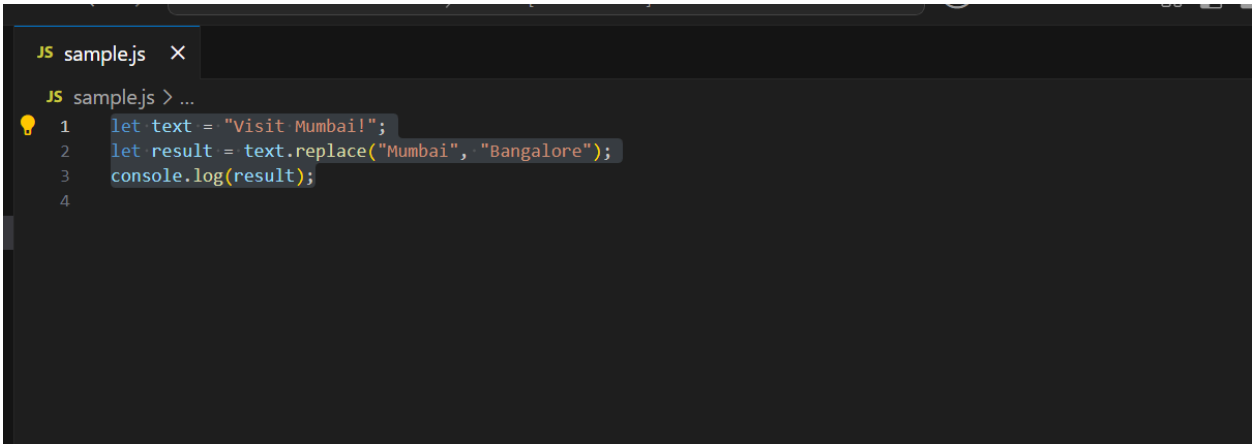


JavaScript String replace()

The `replace()` method searches a string for a value or a regular expression.

The `replace()` method returns a new string with the value(s) replaced.

The `replace()` method does not change the original string.



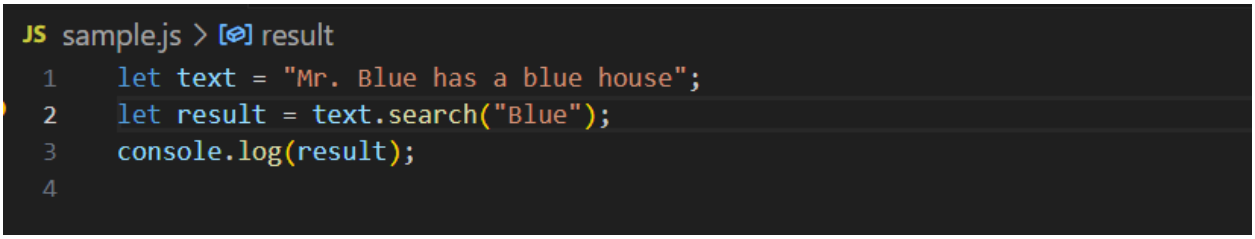
JavaScript String search()

The `search()` method matches a string against a regular expression **

The `search()` method returns the index (position) of the first match.

The `search()` method returns -1 if no match is found.

The `search()` method is case sensitive.



```
// JS Array Methods
```

JavaScript Arrays

An Array is an object type designed for storing data collections.

Key characteristics of JavaScript arrays are:

- Elements: An array is a list of values, known as elements.
- Ordered: Array elements are ordered based on their index.
- Zero indexed: The first element is at index 0, the second at index 1, and so on.
- Dynamic size: Arrays can grow or shrink as elements are added or removed.
- Heterogeneous: Arrays can store elements of different data types (numbers, strings, objects and other arrays).

Why Use Arrays?

If you have a list of items (a list of car names, for example), storing the names in single variables could look like this:

```
let car1 = "Saab";

let car2 = "Volvo";

let car3 = "BMW";
```

However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?

The solution is an array!

An array can hold many values under a single name, and you can access the values by referring to an index number.

Creating an Array

Using an array literal is the easiest way to create a JavaScript Array.

Syntax:

```
const array_name = [item1, item2, ...];
```

Two ways to create Array

```
const cars = [];    // most commonly used
```

```
const cars = new Array("Saab", "Volvo", "BMW");
```

Array methods

Arrays provide a lot of methods. To make things easier, in this chapter, they are split into groups.

Add/remove items

We already know methods that add and remove items from the beginning or the end:

- `arr.push(...items)` – adds items to the end,
- `arr.pop()` – extracts an item from the end,
- `arr.shift()` – extracts an item from the beginning,
- `arr.unshift(...items)` – adds items to the beginning.

```
JS sample.js > ...
1   let array = ["Hello", "Green", "Orange"];
2
3   array.push("World");
4   console.log(array);
5
6   array.pop();
7   console.log(array);
8
9   array.shift();
10  console.log(array);
11  💡
12  array.unshift("Hello");
13  console.log(array);
14
```

JavaScript Array concat()

The `concat()` method creates a new array by merging (concatenating) existing arrays:

```
JS sample.js ×
JS sample.js > ...
1   const arr1 = ["Cecilie", "Lone"];
2   const arr2 = ["Emil", "Tobias", "Linus"];
3   console.log(arr1.concat(arr2));
4
```

Sorting an Array

The `sort()` method sorts an array alphabetically:

Reversing an Array

The `reverse()` method reverses the elements in an array:


```
//JS Array methods end
```

```
// JS functions
```

Functions

Functions in JavaScript are reusable blocks of code designed to perform specific tasks. They allow you to organize, reuse, and modularize code. It can take inputs, perform actions, and return outputs.

Understanding Functions / Function Declaration

In functions, **parameters** are placeholders defined in the function, while **arguments** are the actual values you pass when calling the function

Example:

```
function greet(name) {    // 'name' is a parameter
 console.log("Hello " + name);
}

greet("Alice");  // "Alice" is the argument
```

- Parameter → name (placeholder inside the function).
- Argument → "Alice" (real value given at call time).

JavaScript Function Expression

A **function expression** is a way to define a function as part of an expression making it versatile for assigning to variables, passing as arguments, or invoking immediately.

- Function expressions can be **named** or **anonymous**.
- They are **not** **hoisted**, meaning they are accessible only after their definition.
- Frequently used in **callbacks**, **closures**, and dynamic function creation.
- Enable **encapsulation** of functionality within a limited scope.

```
const greet = function(name) {
   return `Hello, ${name}!`;
};
console.log(greet("Ananya"));
```

Output

```
Hello, Ananya!
```

```
// JS Function ends

// JS Operators
```

JavaScript Operators

JavaScript operators are special symbols or keywords that perform operations on values and variables (operands). They are fundamental to manipulating data and controlling program flow in JavaScript.

Arithmetic Operators: Perform mathematical calculations.

- `+` (Addition)
- `-` (Subtraction)
- `*` (Multiplication)
- `/` (Division)
- `%` (Modulo - remainder of division)
- `**` (Exponentiation)
- `++` (Increment)
- `--` (Decrement)

Assignment Operators: Assign values to variables.

- `=` (Assignment)
- `+=`, `-=`, `*=`, `/=`, `%=`, `**=` (Compound assignment operators)

Comparison Operators: Compare two values and return a boolean result (`true` or `false`).

- `==` (Loose equality)
- `===` (Strict equality)
- `!=` (Loose inequality)
- `!==` (Strict inequality)
- `>` (Greater than)
- `<` (Less than)
- `>=` (Greater than or equal to)
- `<=` (Less than or equal to)

Logical Operators: Combine or modify boolean expressions.

- `&&` (Logical AND)
- `||` (Logical OR)
- `!` (Logical NOT)

// JS Objects

Objects

In JavaScript, an object is a fundamental data structure that allows you to store collections of related data and functionality. It can be viewed as a container for key-value pairs, where:

Keys (Property Names): These are unique identifiers for the values within the object. They must be strings or Symbols.

Values (Property Values): These can be any JavaScript data type, including primitive values (numbers, strings, booleans, null, undefined, symbols), other objects, or functions. When a property's value is a function, it is referred to as a method.

An empty object ("empty cabinet") can be created using one of two syntaxes:

```
let user = new Object(); // "object constructor" syntax
let user = {};  // "object literal" syntax
```

We can immediately put some properties into `{...}` as "key: value" pairs:

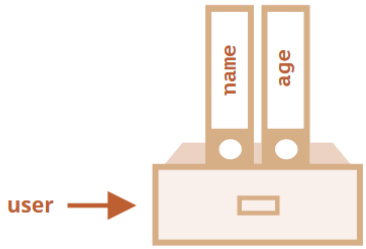
```
let user = {     // an object
  name: "John",  // by key "name" store value "John"
  age: 30        // by key "age" store value 30
};
```

A property has a key (also known as "name" or "identifier") before the colon `":"` and a value to the right of it.

In the `user` object, there are two properties:

1. The first property has the name `"name"` and the value `"John"`.
2. The second one has the name `"age"` and the value `30`.

The resulting `user` object can be imagined as a cabinet with two signed files labeled “name” and “age”.



We can add, remove and read files from it at any time.

Property values are accessible using the dot notation:

JS sample.js

```
1  console.log(user.name); // John
2  console.log(user.age); // 30
3
```

The "for..in" loop

To walk over all keys of an object, there exists a special form of the loop: `for..in`. This is a completely different thing from the `for(;;)` construct that we studied before.

The syntax:

```
for (key in object) {
  // executes the body for each key among object properties
}
```

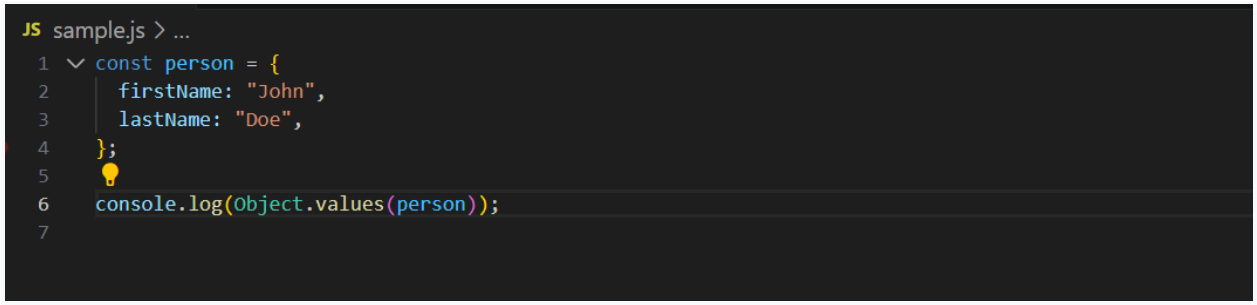
For instance, let's output all properties of `user`:

```
let user = {
  name: "John",
  age: 30,
  isAdmin: true
};

for (let key in user) {
  // keys
  alert( key );  // name, age, isAdmin
  // values for the keys
  alert( user[key] ); // John, 30, true
}
```

Object Methods

- `Object.keys(obj)`: Returns an array of a given object's own enumerable property names.

- `Object.values(obj)`: Returns an array of a given object's own enumerable property values.



- `Object.assign(target, source)`: Copies properties from a source object to a target object.

```
JS sample.js > email
1  const person = {
2    firstName: "John",
3    lastName: "Doe",
4  };
5
6  console.log(Object.assign(person, { email: "email" }));
7
```

This keyword

`this` keyword in JavaScript is a special identifier that refers to the object currently executing the function.

`this` keyword always refer to current object context

```
JS sample.js > ...
1  const user = {
2    name: "Alice",
3    greet: function () {
4      console.log(`Hello, my name is ${this.name}`);
5    },
6  };
7  user.greet(); // Output: Hello, my name is Alice (this refers to 'user')
8
```

Nested Objects:

In JavaScript, a nested object is an object that is a property of another object. This allows for the creation of hierarchical and complex data structures, where data can be organized in a more structured and logical manner.

JavaScript

```javascript
const person = {
  name: "Alice",
  age: 30,
  address: {
    street: "123 Main St",
    city: "Anytown",
    zipCode: "12345"
  },
  contact: {
    email: "alice@example.com",
    phone: "555-123-4567"
  }
};
```

In this example:

- `person` is the main object.
- `address` and `contact` are nested objects within the `person` object.
- `address` has properties like `street`, `city`, and `zipCode`.
- `contact` has properties like `email` and `phone`.

Accessing Nested Properties:

Properties within nested objects are accessed using dot notation or bracket notation, chaining them together to traverse the hierarchy.

JavaScript

```
// Using dot notation
console.log(person.address.city); // Output: Anytown
console.log(person.contact.email); // Output: alice@example.com

// Using bracket notation
console.log(person["address"]["street"]); // Output: 123 Main St
console.log(person["contact"]["phone"]); // Output: 555-123-4567
```

JSON

JSON stands for JavaScript Object Notation.

JSON is a plain text format for storing and transporting data.

JSON is similar to the syntax for creating JavaScript objects.

JSON is used to send, receive and store data.

JSON.parse()

A common use of JSON is to exchange data to/from a web server.

When receiving data from a web server, the data is always a string.

Parse the data with `JSON.parse()`, and the data becomes a JavaScript object.

JSON.stringyfy()

A common use of JSON is to exchange data to/from a web server.

When sending data to a web server, the data has to be a string.

You can convert any JavaScript datatype into a string with `JSON.stringify()`.

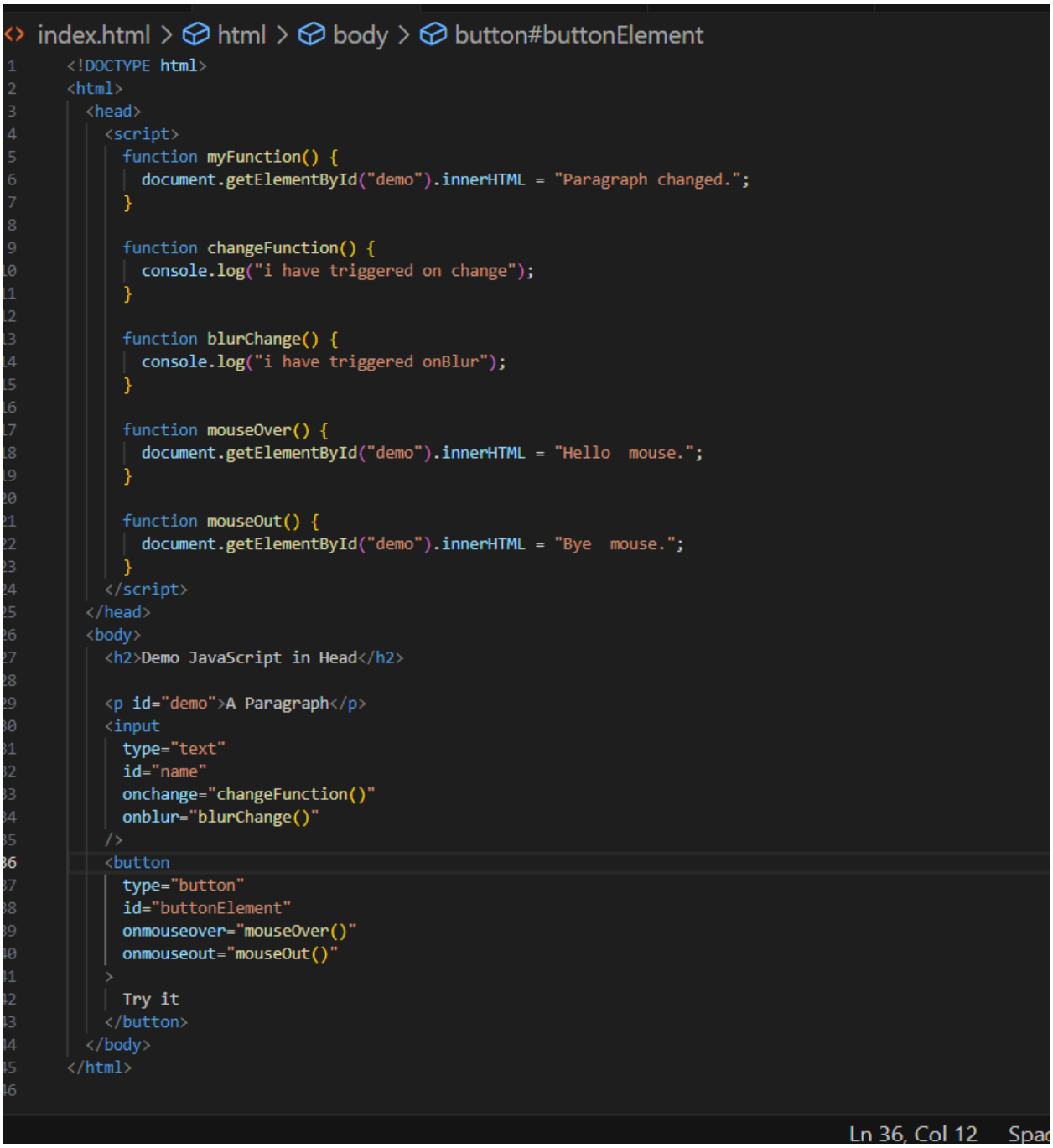
const obj = {name: "John", age: 30, city: "New York"};

JS events

Common types of JavaScript events include:

- Mouse Events: These are triggered by mouse interactions, such as `click`, `dblclick`, `mouseover`, `mouseout`, `mousedown`, `mouseup`, and `mousemove`.

- Keyboard Events: These respond to keyboard interactions, including `keydown`, `keypress`, and `keyup`.
- Form Events: These are related to form elements and actions, such as `submit`, `change`, `focus`, `blur`, and `input`.



JS Template literals:

Template literals, introduced in ES6 (ECMAScript 2015), are a powerful feature in JavaScript that provide a more flexible and readable way to work with strings compared to traditional string concatenation.

Key Features:

- Backticks (`` ` ``): Template literals are enclosed in backticks, not single or double quotes.
- String Interpolation: You can embed variables and expressions directly within the string using the `${expression}` syntax. This eliminates the need for string concatenation operators (`+`).

JavaScript

```
const name = "Alice";
const age = 30;
const greeting = `Hello, ${name}! You are ${age} years old.`;
console.log(greeting); // Output: Hello, Alice! You are 30 years old.
```

- **Multi-line Strings:** Template literals naturally support multi-line strings without needing escape characters (`\n`).

JavaScript

```
const multiLineString = `This is the first line.
This is the second line.
And this is the third line.`;
console.log(multiLineString);
```

- **Embedded Expressions:** Any valid JavaScript expression can be placed inside the `${}` placeholder, including function calls, arithmetic operations, and conditional (ternary) operators.

JavaScript

```
const x = 5;
const y = 10;
const result = `The sum of ${x} and ${y} is ${x + y}.`;
console.log(result); // Output: The sum of 5 and 10 is 15.

const isAdult = true;
const status = `You are ${isAdult ? 'an adult' : 'a minor'}.`;
console.log(status); // Output: You are an adult.
```

JS Type conversion

Type conversion in JavaScript refers to the process of converting a value from one data type to another. This can happen either automatically (implicit conversion or coercion) or manually (explicit conversion or type casting).

Implicit Type Conversion

Implicit conversion occurs automatically by JavaScript when performing operations or comparisons involving different data types. JavaScript attempts to make sense of the operation by converting one or both operands to a compatible type.

Examples:

- **String Concatenation:** When a number is added to a string using the `+` operator, the number is implicitly converted to a string.

JavaScript

```
let result = "Hello " + 123; // result becomes "Hello 123" (string)
```

- **Arithmetic Operations:** When a string representing a number is used in an arithmetic operation (except `+`), it's often implicitly converted to a number.

JavaScript

```
let result = "10" * 2; // result becomes 20 (number)
```

- **Boolean Context:** Values are implicitly converted to booleans in conditional statements or when using logical operators. Falsy values (false, 0, "", null, undefined, NaN) become `false`, while all other values become `true`.

JavaScript

```
if ("hello") { // "hello" is implicitly converted to true
    console.log("This will execute");
}
```

Explicit Type Conversion

Explicit conversion involves manually changing the data type of a value using built-in JavaScript functions or operators. This provides more control over the conversion process.

Methods for Explicit Conversion:

- **To Number:**
  - `Number()` function: Converts a value to a number.

JavaScript

```
let num = Number("123"); // num becomes 123 (number)
let invalidNum = Number("abc"); // invalidNum becomes NaN (number)
```

- `parseInt()` and `parseFloat()`: Parse a string and return an integer or a floating-point number, respectively. They stop parsing at the first non-numeric character.

JavaScript

```
let intVal = parseInt("10.5px"); // intVal becomes 10 (number)
let floatVal = parseFloat("3.14 meters"); // floatVal becomes 3.14 (numb
```

- **To String:**
  - `String()` function: Converts a value to a string.

JavaScript

```
let str = String(123); // str becomes "123" (string)
```

- `.toString()` method: Available on most data types.

JavaScript

```
let num = 42;
let str = num.toString(); // str becomes "42" (string)
```

- **To Boolean:**
  - `Boolean()` function: Converts a value to a boolean.

JavaScript

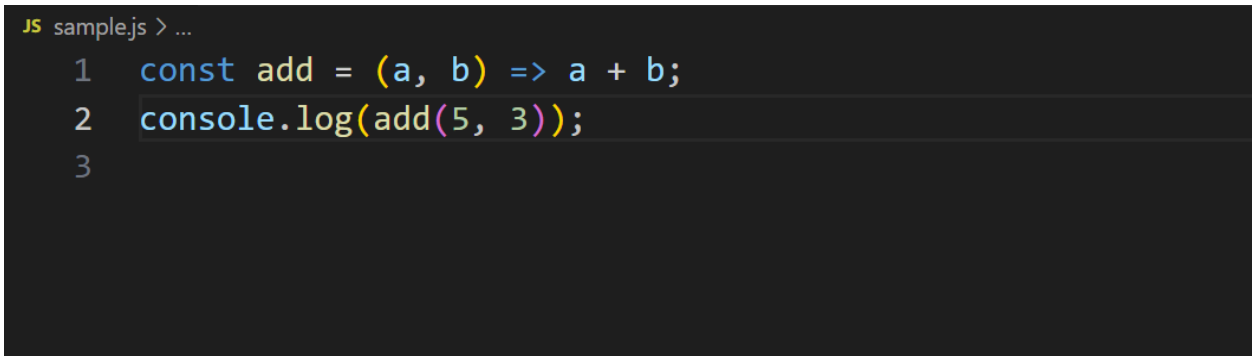
```
let bool1 = Boolean(0); // bool1 becomes false
let bool2 = Boolean("hello"); // bool2 becomes true
```

Understanding both implicit and explicit type conversion is crucial for writing robust and predictable JavaScript code, as it helps in avoiding unexpected behavior and errors.

Arrow Function

Arrow functions in JavaScript, introduced in ES6, provide a concise syntax for writing function expressions.

```
const add = (a, b) => {
  // Multiple statements can go here
  return a + b;
};
```



Lexical Scope

Lexical scope is a fundamental concept in programming that determines the accessibility of variables and functions based on where they are defined in the source code. In simple terms, lexical scope is the scope of a variable or function determined at compile time by its physical location in the code.

- **Global Scope:** Variables defined outside any function or block, accessible anywhere in the program.
- **Local Scope:** Variables defined inside a function or block, accessible only within that specific function or block.
- **Nested Scope:** Inner functions have access to variables in their parent functions.
- **Block Scope:** Variables defined with let and const are limited to the block they are declared in, like loops or conditionals.

Global Scope

When a variable is defined outside of any functions or blocks, it has a global scope. This means that it can be accessed from anywhere within the program, including within functions.

Example: In the example above, the variable name is defined outside of any functions or blocks, making it a global variable. The function sayHello is then able to access the name variable and output "Hello John" to the console.

```
let name = "John"; // Global variable

function sayHello() {
      console.log("Hello " + name);
}

sayHello(); // Output: "Hello John"
```

Local Scope

When a variable is defined within a function or block, it has a local scope. This means that it can only be accessed within that function or block.

Example: In this example, name is a local variable within sayHello. It's accessible inside the function, but attempting to access it outside results in a "ReferenceError.

```
function sayHello() {
      let name = "John"; // Local variable

      console.log("Hello " + name);
}

sayHello(); // Output: "Hello John"

console.log(name);
// Output: Uncaught ReferenceError: name is not defined
```

Nested Scope

When a function is defined within another function, it has access to variables defined in the parent function. This is known as nested scope.

Example: In the example above, the function inner is defined within the outer function. The inner function is able to access the name variable defined in the parent outer function, outputting "Hello John" to the console.

```
function outer() {
    let name = "John"; // Outer function variable

    function inner() {
        console.log("Hello " + name);
    }

    inner(); // Output: "Hello John"
}

outer();
```

Output

```
Hello John
```

Block Scope

ES6 introduced the let and const keywords, which allow variables to have block scope. This means that variables defined within a block of code (such as within an if statement or a for loop) can only be accessed within that block.

Example: In this example, message is a block-scoped variable. It's accessible within the if block, but accessing it outside results in a "ReferenceError.

```
function sayHello() {
    let name = "John"; // Function variable

    if (true) {
        let message = "Hello"; // Block variable
        console.log(message + " " + name);
        // Output: "Hello John"
    }

    console.log(message);
    // Output: Uncaught ReferenceError:
    // message is not defined
}

sayHello();
```

Output:

JS Closure

A closure is a function that retains access to its outer function's variables, even after the outer function has finished executing. It "remembers" the environment in which it was created, allowing it to access variables outside its immediate scope.

Using closure we can achieve state maintenance.

In the above function in the output both the times the count is printed as one.

```
JS sample.js > ...
1  function counter() {
2    // Private variable
3    let count = 0;
4
5    function state() {
6      // Access and modify the private variable
7      count++;
8      console.log("i have a count", count);
9    }
10
11   return state;
12 }
13
14 const increment = counter();
15 increment();
16 increment();
17 increment();
18
```

In this code , value of count is retained even after the counter function is executed once on line 15. .This is make you understand what is closure. (Read definition again)

Classes

Blueprints for creating multiple instances of objects with shared properties and methods.

- **Instance creation:** Objects are created from a class using the `new` keyword followed by the class name and any arguments required by the constructor.

Class example

```js
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    console.log(
      `Hello, my name is ${this.name} and I am ${this.age} years old.`
    );
  }
}

const person1 = new Person("Alice", 30);
person1.greet();
```

Classes Inheritance

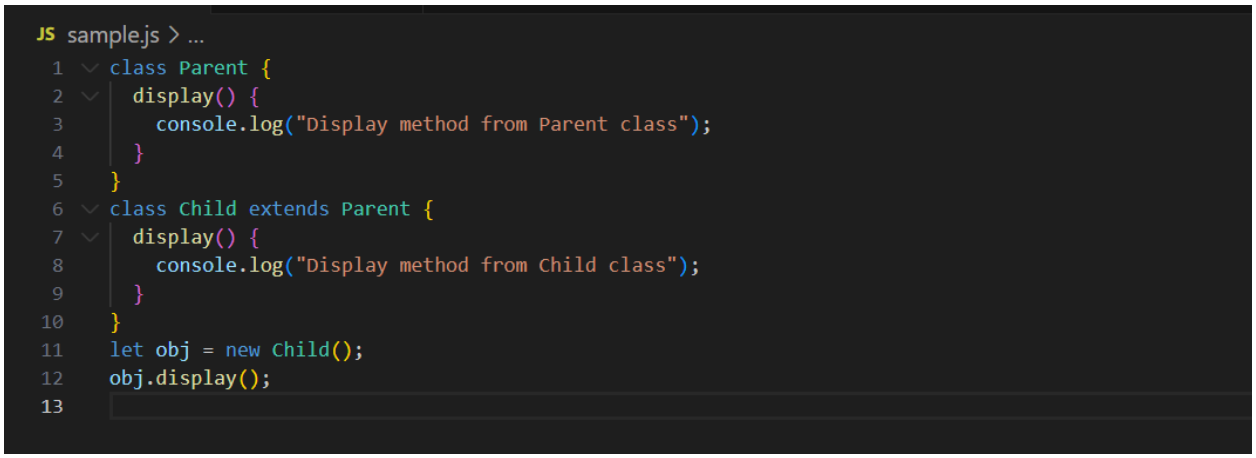
Class inheritance in JavaScript allows a new class (child class or subclass) to inherit properties and methods from an existing class (parent class or superclass). This mechanism promotes code reusability and helps organize code into a hierarchical structure.

```js
class Product {
  constructor(name) {
    this.name = name;
  }
  buyProduct() {
    console.log("i am buying a product");
  }
}
class Mobile extends Product {
  buyMobile() {
    console.log("i have a mobile");
  }
}

const mobile = new Mobile();
mobile.buyProduct();
mobile.buyMobile();
```

Overriding

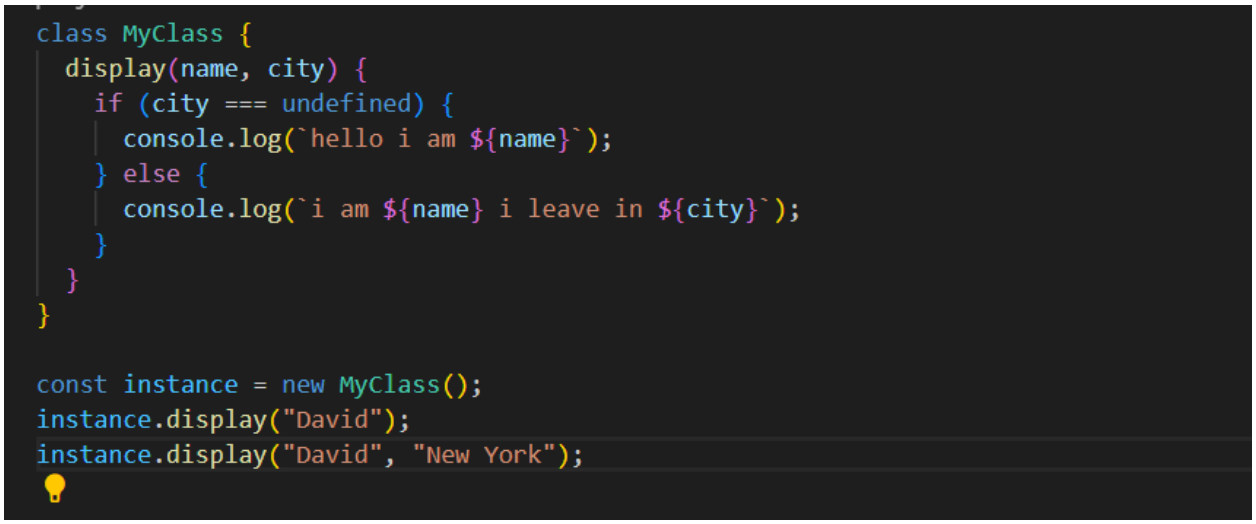
The Function overriding occurs when a subclass or child class provides the specific implementation for the method that is already defined in its superclass or parent class.

This allows a subclass to the tailor the method to its own needs. Function overriding in JavaScript allows a subclass or child class to provide a specific implementation of a method that is already defined in its superclass or parent class. This enables the child class to show the behavior of the inherited method that can be manipulated by the child class itself.



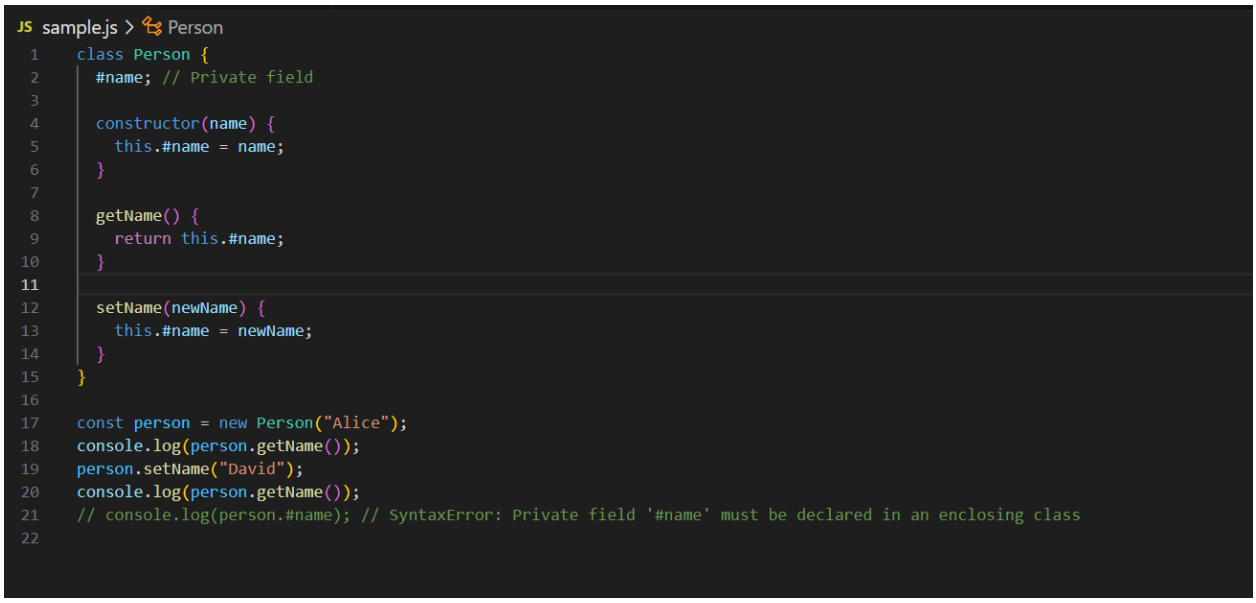
Overloading:

Function Overloading is a feature found in many object-oriented programming languages, where multiple functions can share the same name but differ in the number or type of parameters.



Encapsulation

Encapsulation is a fundamental concept in object-oriented programming that refers to the practice of hiding the internal details of an object and exposing only the necessary information to the outside world.



Polymorphism:

Polymorphism of object-oriented programming languages where **poly** means **many** and **morphism** means **transforming one form into another**. Polymorphism means the same function with different signatures is called many times. It allows methods to do different things based on the object it is acting upon.

sample.js > ...

```js
class Animal {
  speak() {
    console.log("Animal makes a sound");
  }
}

class Dog extends Animal {
  speak() {
    console.log("Dog barks");
  }
}

class Cat extends Animal {
  speak() {
    console.log("Cat meows");
  }
}

const dog = new Dog();
dog.speak();

const cat = new Cat();
cat.speak();
```

Super keyword

The `super` keyword is used to call the constructor of its parent class to access the parent's properties and methods.

JS sample.js > ...

```
class Product {
  constructor(name, color) {
    this.name = name;
    this.color = color;
  }
}

class Apple extends Product {
  constructor(name, color, storage) {
    super(name, color);
    this.storage = storage;
  }

  buyMobile() {
    console.log(
      `i am buying ${this.name} of ${this.color} color with ${this.storage} GB`
    );
  }
}

class Samsung extends Product {
  constructor(name, color, storage) {
    super(name, color);
    this.storage = storage;
  }

  buy() {
    console.log(
      `i am buying ${this.name} of ${this.color} color with ${this.storage} GB`
    );
  }
}

const apple = new Apple("Apple", "Red", "200");
apple.buyMobile();
const samsung = new Samsung("Samsung", "Blue", "200");
samsung.buy();
```

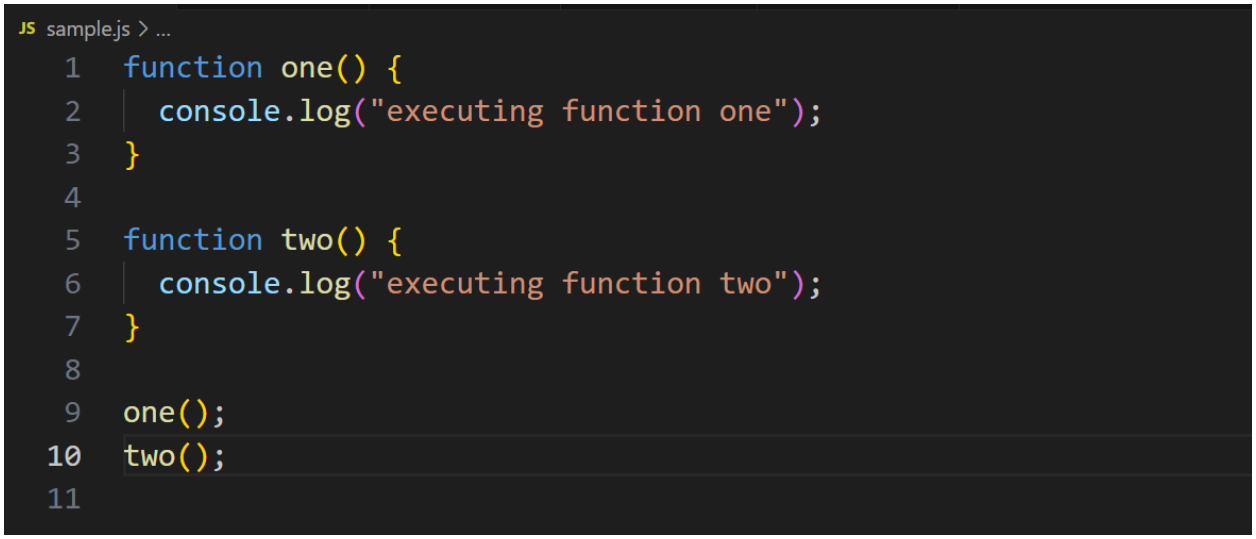
JS sample.js ✕ index.html ●

JS sample.js > ...

```
class Parent {
  greet() {
    return "Hello from Parent!";
  }
}

class Child extends Parent {
  greet() {
    return `${super.greet()} And hello from Child!`; // Calls Parent's greet method
  }
}

const childInstance = new Child();
console.log(childInstance.greet()); // Output: Hello from Parent! And hello from Child!
```

JS Callbacks

In JavaScript, a callback function is a function that is passed as an argument to another function and is intended to be executed at a later time, typically after the completion of some operation within the outer function. This mechanism is fundamental for handling asynchronous operations and for extending the functionality of higher-order functions.



In the above code , functions are executed in order.

```
JS sample.js > one > setTimeout() callback
1  function one() {
2    setTimeout(function () {
3      console.log("executing function one");
4    }, 3000);
5  }
6
7  function two() {
8    console.log("executing function two");
9  }
10
11 one();
12 two();
13
```

In the above function , function two is executed first and then function one . this was due to the timeout or time taken by function one to respond is 3 sec.

To handle such asynchronous scenarios we add a callback.

```
JS sample.js > ...
1  function one(callback) {
2    setTimeout(function () {
3      console.log("executing function one");
4      callback();
5    }, 3000);
6  }
7
8  function two() {
9    console.log("executing function two");
10 }
11
12 one(two);
13
```

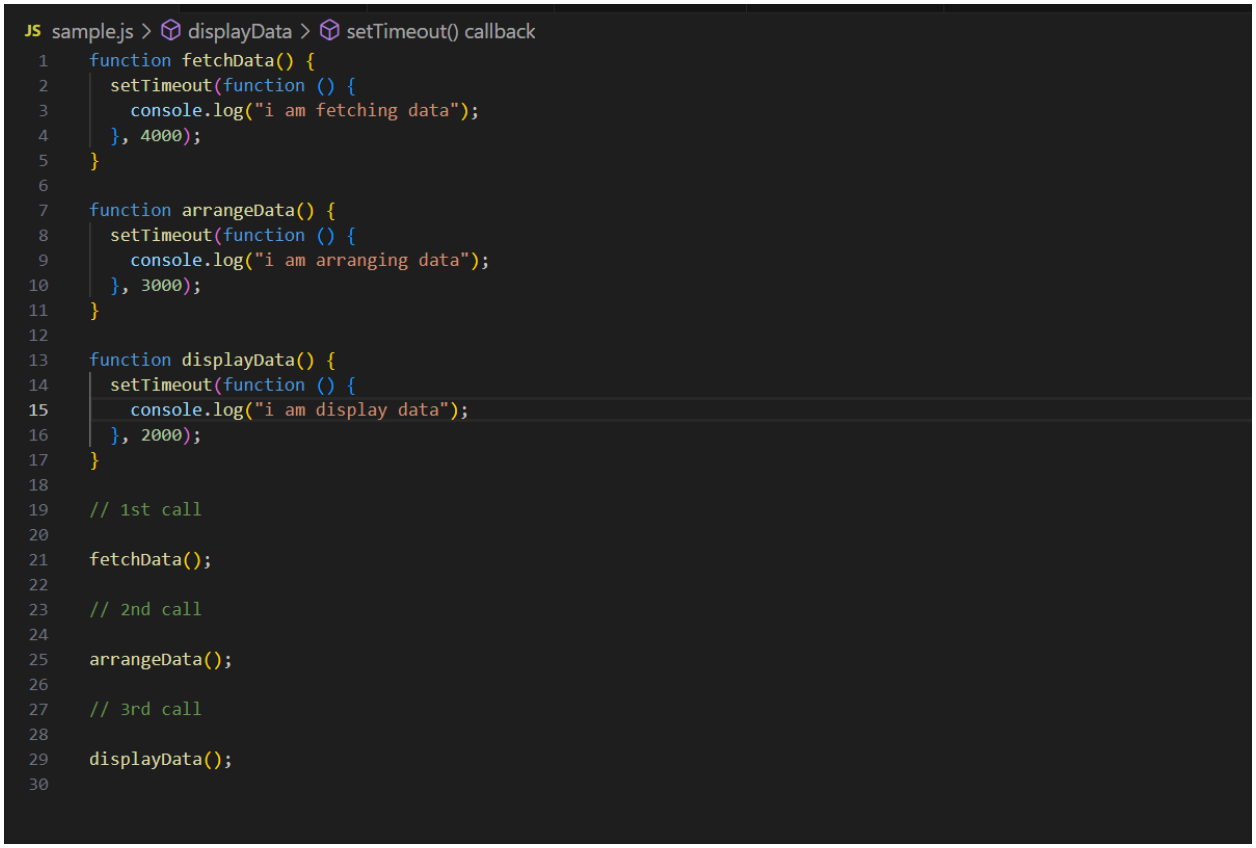
Use of writing a call back : In the below program , displayResult function execution has a meaning if it has result value and result value is captured in performOperation function which executes asynchronously . So we have to wait for performOperation execution to complete and then execute displayResult

```
function performOperation(value, callback) {
  console.log("Performing operation with:", value);
  // Simulate an asynchronous operation
  setTimeout(() => {
    const result = value * 2;
    callback(result); // Execute the callback with the result
  }, 1000);
}

function displayResult(data) {
  console.log("The operation result is:", data);
}


performOperation(5, displayResult);
```

Lets take one more scenarios to understand callback

- Fetch Data
- Arrange Data
- Display Data

Program for above scenario



If we execute , we get output ,

I am display data

I am arranging data

I am fetching data

But to my requirement execution order is not working as expected , hence to execute to functions in proper order we will introduce callback .

Callback : Passing function as argument to another function and to decided order of execution we use callbacks

Callback hell:

Callback hell is a situation in asynchronous programming where multiple callbacks are nested within each other, creating a "pyramid of doom" structure that is difficult to read, maintain, and debug. It occurs when one callback function calls another, which calls another, and so on, making the code's indentation and logic complex.

JS Promise

A Promise in JavaScript is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. It provides a cleaner and more

structured way to handle asynchronous code compared to traditional callbacks, helping to avoid "callback hell."

A Promise can exist in one of three states:

- **Pending:** The initial state; the asynchronous operation has not yet completed.
- **Fulfilled (Resolved):** The operation completed successfully, and the promise now holds a resulting value.
- **Rejected:** The operation failed, and the promise holds an error object as the reason for failure.

Sample promise example

Handling Promise Outcomes:

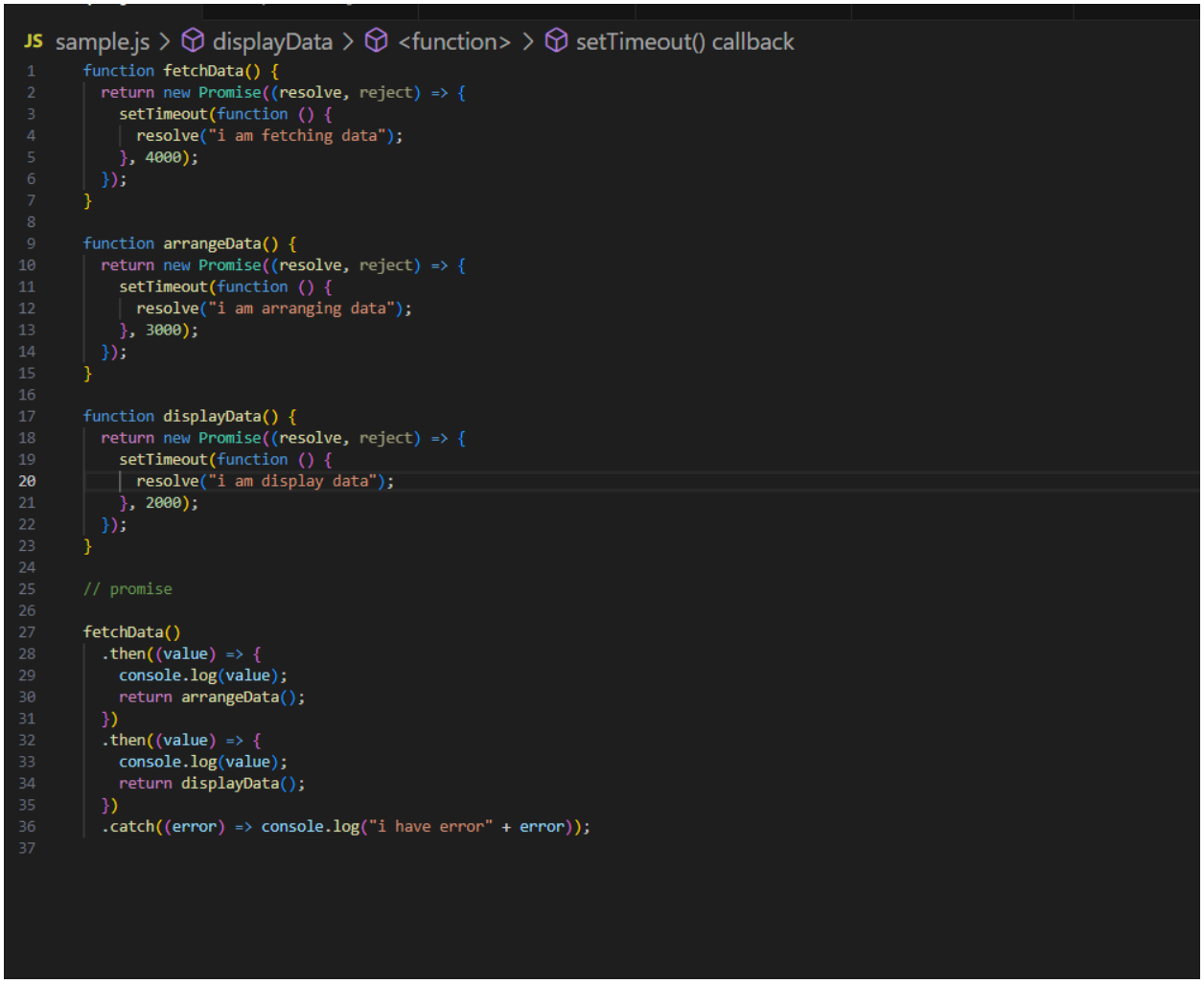
Promises use `.then()`, `.catch()`, and `.finally()` methods to handle their eventual outcomes:

- `.then(onFulfilled, onRejected)`: Attaches handlers for when the promise is fulfilled (`onFulfilled`) or rejected (`onRejected`). The `onRejected` handler is optional.
- `.catch(onRejected)`: A shorthand for `.then(null, onRejected)`, specifically for handling rejections (errors).
- `.finally(onFinally)`: Attaches a handler that executes regardless of whether the promise was fulfilled or rejected. It does not receive any arguments and its return value is generally ignored.

Promise Chaining:

Promises can be chained together, allowing for sequential execution of asynchronous operations. Each `.then()` or `.catch()` returns a new promise, enabling further chaining.

Lets execute callback program with promise now



Async / await

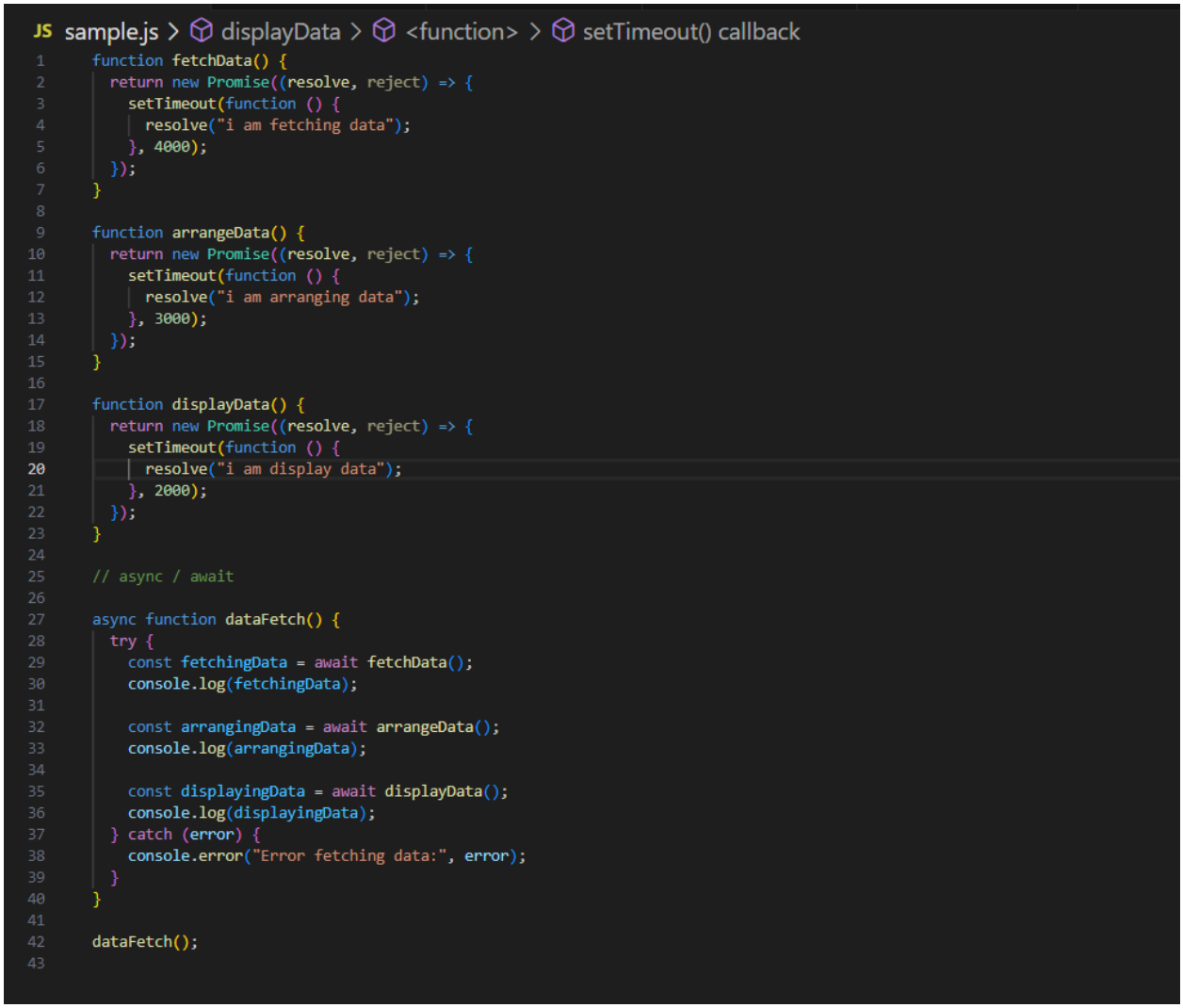
`async` and `await` in JavaScript provide a more synchronous-looking syntax for handling asynchronous operations, built on top of Promises.

`async` keyword:

- The `async` keyword is placed before a function declaration to mark it as an asynchronous function.
- An `async` function always implicitly returns a Promise. If the function's return value is not explicitly a Promise, it will be automatically wrapped in a `Promise.resolve()`.

`await` keyword:

- The `await` keyword can only be used inside an `async` function.
- It pauses the execution of the `async` function until the Promise it's waiting for is settled (either fulfilled or rejected).
- If the Promise resolves successfully, `await` returns the resolved value.
- If the Promise is rejected, `await` throws the rejected value as an error, which can be caught using a `try...catch` block.



Destructuring:

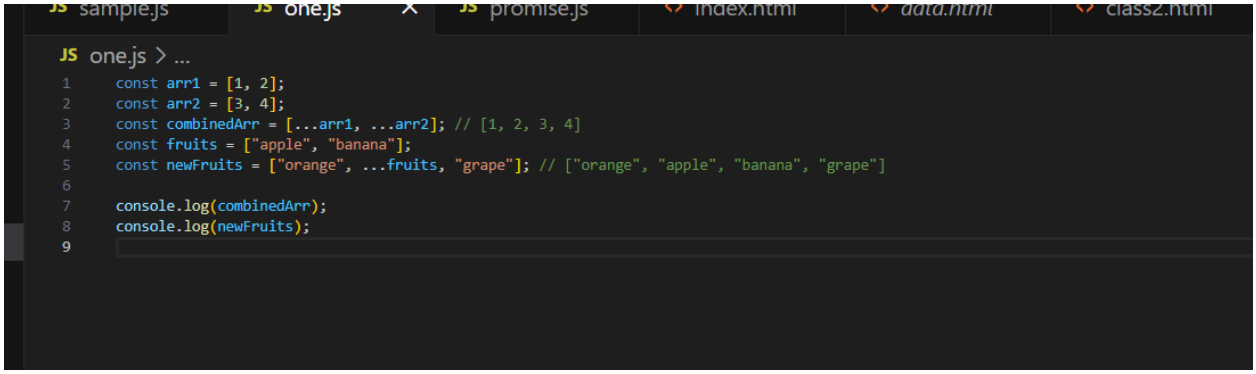
Destructuring in JavaScript is a powerful feature that enables the extraction of values from arrays or properties from objects into distinct variables. This syntax simplifies the process of accessing and assigning data from complex data structures.

Hoisting:

Hoisting in JavaScript is a behavior where variable and function declarations are conceptually moved to the top of their containing scope during the compilation phase, before the code is executed. This means that you can use a variable or call a function before it is explicitly declared in your code.

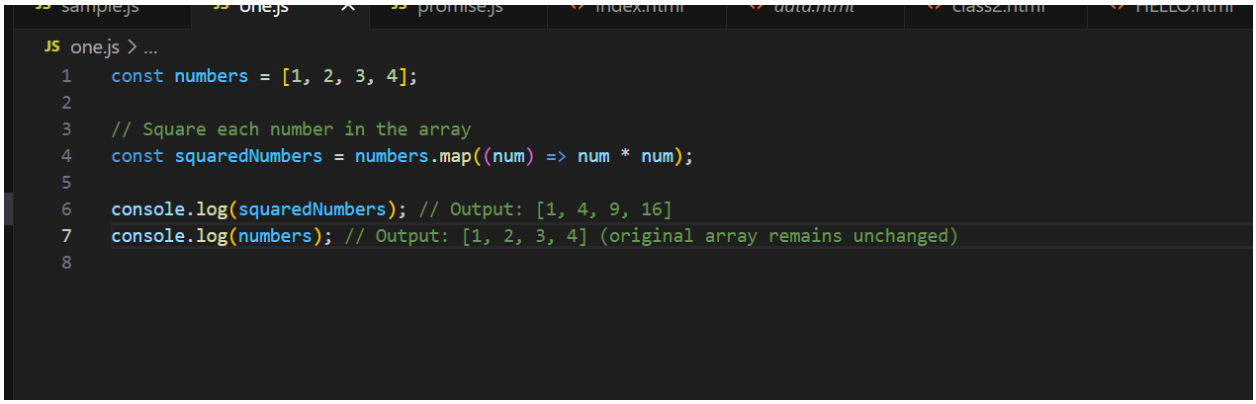
Spread Operator

The spread operator in JavaScript, denoted by three dots (`...`), is a powerful syntax introduced in ES6 that allows you to expand an iterable (like an array or string) or an object into individual elements or properties.



JS map and filter method

The `map()` method in JavaScript is a higher-order array method used to create a new array by applying a provided function to each element of the original array. It iterates through each element, executes a callback function for each, and collects the results into a new array. The original array remains unchanged.



The `filter()` method in JavaScript is an immutable array method used to create a new array containing only the elements from the original array that satisfy a specified condition. It does not modify the original array.

JS one.js > ...

```js
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// Filter out even numbers
const evenNumbers = numbers.filter((number) => number % 2 === 0);
console.log(evenNumbers); // Output: [2, 4, 6, 8, 10]

// Filter out numbers greater than 5
const greaterThanFive = numbers.filter((number) => number > 5);
console.log(greaterThanFive); // Output: [6, 7, 8, 9, 10]

```

JS one.js > ...

```js
const users = [
  { name: "Alice", city: "London" },
  { name: "Bob", city: "Paris" },
  { name: "Charlie", city: "London" },
  { name: "David", city: "New York" },
];

const usersInLondon = users.filter((user) => user.city === "London");

console.log(usersInLondon);

```